

ENGINEERING GUIDE

Flutter CI/CD & DevOps

Choosing your tooling, building the pipeline, surviving code signing, and shipping to both stores without losing your weekend.

Covers: GitHub Actions · Codemagic · Fastlane · Bitrise · GitLab CI

Signing: fastlane match · App Store Connect API · Play App Signing

Prepared: 12 August 2026

Contents

1. The Landscape — and the mistake everyone makes

Why "Codemagic vs Fastlane" is the wrong question

2. Choosing Your CI Provider

Comparison table, cost reality, and a straight recommendation

3. Pipeline Architecture

Three tiers, not one giant job

4. GitHub Actions — Complete Configuration

Working YAML for test, Android, and iOS jobs

5. Code Signing — The Part That Actually Hurts

Android keystores, iOS certificates, fastlane match

6. Codemagic — The Low-Pain Alternative

Same outcome, a fraction of the config

7. Configuration, Flavors & Secrets

dart-define, environments, and what must never reach the binary

8. Versioning Strategy

Let CI own the build number

9. Distribution & Release Channels

Firebase App Distribution, TestFlight, Play tracks, Shorebird

10. Testing in CI

Unit, widget, integration, and E2E — and what to gate

11. Code Quality & Repo Hygiene

Linting, formatting, dependencies, monorepos

12. Monitoring & Crash Reporting

Symbol uploads, or your stack traces are useless

13. Performance & Cost Optimisation

Caching, runner selection, minute budgeting

14. Common Pitfalls

The failures that cost people days

15. Implementation Roadmap

A four-week plan and a printable checklist

1. The Landscape — and the mistake everyone makes

Almost every recommendation you'll get lists "Codemagic, Fastlane, Bitrise" as if they're alternatives to each other. They aren't. Understanding the layers is the single most clarifying thing you can do before picking anything.

Three distinct layers

Layer 1 — CI Provider · GitHub Actions, Codemagic, Bitrise, GitLab CI, CircleCI

Rents you a Linux or macOS machine. Watches your git repo. Runs your YAML on push. This is the layer you choose.

Layer 2 — Mobile Task Runner · Fastlane (Ruby), or Codemagic CLI Tools (Python)

Runs *inside* the CI machine. Handles certificates, provisioning profiles, store uploads, screenshot generation, changelog management. Not a CI system.

Layer 3 — Flutter Toolchain · flutter build, flutter test, flutter analyze

The actual compilation and testing. Identical regardless of what sits above it.

THE KEY INSIGHT

Fastlane is not a CI provider. Codemagic is not a task runner. You will typically use one from Layer 1 *plus* one from Layer 2. The notable exception: Codemagic has built-in signing and publishing, so it can replace Fastlane entirely — which is precisely why people recommend it to Flutter developers.

What each tool actually does

Tool	Layer	What it gives you
GitHub Actions	CI provider	Linux + macOS runners, tight GitHub integration, huge marketplace of actions, YAML in your repo
Codemagic	CI provider (+ signing)	Flutter-first CI with built-in code signing UI, automatic store publishing, M2 Mac machines
Bitrise	CI provider	GUI-driven step library, strong enterprise features, mobile-focused
GitLab CI	CI provider	Built into GitLab, supports self-hosted runners (including your own Mac mini)
Fastlane	Task runner	Certificate sync (<code>match</code>), TestFlight/Play uploads, build orchestration, screenshots
Shorebird	Delivery	Over-the-air Dart code push — patch a bug without a store review
Melos	Repo tooling	Monorepo management for multi-package Flutter projects

2. Choosing Your CI Provider

Comparison

Provider	Pick it when	Watch out for	Free tier
GitHub Actions	Already on GitHub · want one CI for backend and app · want full control and portability · comfortable wiring signing yourself	macOS runners bill at a 10× minute multiplier on private repos. You build the signing plumbing.	2,000 min/mo on private repos — which is only ~200 macOS minutes . Unlimited on public repos.
Codemagic	Solo dev or small team · want builds shipping this week · iOS signing is eating your life	Costs money past the free tier. Some lock-in to their UI-managed credentials.	500 min/mo on M2 machines.
Bitrise	Larger team · want a visual step library · non-engineers touching pipelines	Pricier. More lock-in. Less Flutter-specific than Codemagic.	Limited trial, then paid.
GitLab CI	You're already a GitLab shop	You'll want a self-hosted macOS runner. Shared macOS runners are limited.	400 min/mo, plus unlimited on self-hosted runners.
Xcode Cloud	Don't, for Flutter	iOS-only and fights Flutter's build system. Useful only for pure-native iOS.	25 h/mo with a developer account.

The recommendation

BOTTOM LINE

Start with GitHub Actions + Fastlane. It's free for Android work, portable, and everything you learn transfers. Move iOS to Codemagic if signing starts costing you days instead of hours — that migration is cheap, and it's exactly the problem Codemagic exists to solve.

Decision shortcuts

- **Solo developer, need TestFlight builds this week** → Codemagic. The 500 free minutes will likely cover you, and the signing UI removes the hardest problem entirely.
- **Team of 3–10, mixed backend + mobile** → GitHub Actions for everything, Fastlane for the mobile chores. One mental model, one place for secrets.
- **Enterprise / regulated / source can't leave the building** → GitLab CI with a self-hosted Mac mini runner. A refurbished M2 Mac mini is a serious, cost-effective option and pays for itself in months versus cloud macOS minutes.
- **Open-source Flutter package** → GitHub Actions, no contest. macOS minutes are free on public repos.

THE COST TRAP

The number that catches everyone out is the macOS multiplier. A 12-minute iOS build on a private GitHub repo consumes **120 minutes** of your 2,000-minute allowance. Run that on every push to every branch and you exhaust a month's budget in about two days. **Only build iOS on `main` and on tags.** This single rule is worth more than any other optimisation in this document.

3. Pipeline Architecture

Do not build one monolithic job that runs on everything. Split into three tiers with different triggers, different runners, and different costs.

Tier 1 — Pull request check

Trigger: every PR, every push to a PR · **Runner:** Ubuntu (cheap, 1× multiplier) · **Target:** under 4 minutes

```
dart format --set-exit-if-changed .  
flutter analyze --fatal-infos  
flutter test --coverage
```

This is your quality gate. It must be fast, because a slow PR check is a PR check people learn to ignore. Never put an iOS build here.

Tier 2 — Merge to main

Trigger: push to `main` · **Runner:** Ubuntu for Android, macOS for iOS · **Target:** under 15 minutes

Builds signed artifacts and ships them to internal testers — Firebase App Distribution for Android, TestFlight internal group for iOS. The goal is that anyone on the team can install the latest `main` on their device without asking a developer.

Tier 3 — Tagged release

Trigger: git tag matching `v*.*.*` · **Runner:** both · **Target:** under 25 minutes

- Release builds with the version derived from the tag
- Upload to Play Console (internal or production track) and App Store Connect
- Upload debug symbols / dSYMs to Crashlytics or Sentry
- Create a GitHub Release with generated changelog and attached artifacts

BUILD IT IN THIS ORDER

Get **Tier 1 green first** and live with it for a week. Then add **Tier 2 Android only** — it's roughly ten times simpler than iOS and proves the whole shape works. Then Tier 2 iOS. Then Tier 3. Attempting all of this in one sitting is the most common way these projects stall.

Trigger matrix

Event	Analyze + Test	Android build	iOS build	Store upload
PR opened / updated	✔	—	—	—
Push to <code>main</code>	✔	✔ → Firebase	✔ → TestFlight	—
Tag <code>v1.2.3</code>	✔	✔	✔	✔ Both stores
Nightly (cron)	✔ + E2E	optional	—	—

4. GitHub Actions — Complete Configuration

A working three-job pipeline. Drop this in `.github/workflows/ci.yml` and adjust the identifiers.

The test job

`.github/workflows/ci.yml`

```
name: ci

on:
  pull_request:
  push:
    branches: [main]
    tags: ['v*.*.*']

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: subosito/flutter-action@v2
        with:
          flutter-version: '3.35.x' # PIN THIS. Never use 'stable'.
          cache: true               # saves ~90s per run

      - run: flutter pub get
      - run: dart format --set-exit-if-changed .
      - run: flutter analyze --fatal-infos
      - run: flutter test --coverage

      - uses: codecov/codecov-action@v4
        with:
          files: coverage/lcov.info
          token: ${ secrets.CODECOV_TOKEN }
```

The Android job

```
android:
  needs: test
  if: github.ref == 'refs/heads/main' || startsWith(github.ref, 'refs/tags/')
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4

    - uses: actions/setup-java@v4
      with:
        distribution: zulu
        java-version: '17'
        cache: gradle

    - uses: subosito/flutter-action@v2
      with: { flutter-version: '3.35.x', cache: true }

    - name: Decode keystore and write signing config
      run: |
        echo "${{ secrets.KEYSTORE_BASE64 }}" | base64 -d > android/app/upload.jks
        cat > android/key.properties <<EOF
        storeFile=upload.jks
        storePassword=${{ secrets.STORE_PASSWORD }}
        keyAlias=${{ secrets.KEY_ALIAS }}
        keyPassword=${{ secrets.KEY_PASSWORD }}
        EOF

    - run: flutter pub get
    - run: flutter build appbundle --release
      --build-number=${{ github.run_number }}

    - name: Upload to Play internal track
      uses: r0adkll/upload-google-play@v1
      with:
        serviceAccountJsonPlainText: ${ secrets.PLAY_SERVICE_ACCOUNT_JSON }
        packageName: com.yourco.app
        releaseFiles: build/app/outputs/bundle/release/app-release.aab
        track: internal
        status: completed
```

The iOS job

```
ios:
  needs: test
  if: github.ref == 'refs/heads/main' || startsWith(github.ref, 'refs/tags/')
  runs-on: macos-15
  steps:
    - uses: actions/checkout@v4

    - uses: subosito/flutter-action@v2
      with: { flutter-version: '3.35.x', cache: true }

    - name: Cache CocoaPods
      uses: actions/cache@v4
      with:
        path: ios/Pods
        key: pods-${{ hashFiles('ios/Podfile.lock') }}

    - run: flutter pub get

    - name: Set up SSH for fastlane match repo
      uses: webfactory/ssh-agent@v0.9.0
      with:
        ssh-private-key: ${ secrets.MATCH_SSH_KEY }

    - name: Build and ship to TestFlight
      working-directory: ios
      env:
        MATCH_PASSWORD: ${ secrets.MATCH_PASSWORD }
        ASC_KEY_ID:      ${ secrets.ASC_KEY_ID }
        ASC_ISSUER_ID:   ${ secrets.ASC_ISSUER_ID }
        ASC_KEY_P8:      ${ secrets.ASC_KEY_P8 }
        BUILD_NUMBER:    ${ github.run_number }
      run: bundle exec fastlane beta
```

TWO DETAILS PEOPLE MISS

Pin the Flutter version. Using `stable` means the Flutter team can break your build with a release you didn't ask for, at the worst possible moment. Pin it, and bump it deliberately in its own PR.

Guard the iOS job with `if:`. Without that condition it runs on every PR push, at a 10× billing multiplier. This is the difference between a free pipeline and an expensive one.

5. Code Signing — The Part That Actually Hurts

Roughly eighty percent of the time people lose on mobile CI goes into code signing. Android is a manageable afternoon. iOS is where projects stall. Here is what actually works.

Android — the straightforward one

Step 1: Enable Play App Signing

Let Google hold the real app signing key. You keep only an *upload key*. This matters enormously: if you lose your upload key, Google can reset it for you. If you were holding the app signing key yourself and lost it, **you could never update that app again** — you'd have to publish under a new package name and abandon your entire install base.

Step 2: Generate and encode the upload key

```
keytool -genkey -v -keystore upload.jks \
  -keyalg RSA -keysize 2048 -validity 10000 -alias upload

base64 -i upload.jks | pbcopy  # paste into GitHub secret KEYSTORE_BASE64
```

Step 3: Wire Gradle to read it

android/app/build.gradle

```
def keystoreProperties = new Properties()
def keystorePropertiesFile = rootProject.file('key.properties')
if (keystorePropertiesFile.exists()) {
    keystoreProperties.load(new FileInputStream(keystorePropertiesFile))
}

android {
    signingConfigs {
        release {
            keyAlias      keystoreProperties['keyAlias']
            keyPassword    keystoreProperties['keyPassword']
            storeFile      keystoreProperties['storeFile']
                        ? file(keystoreProperties['storeFile']) : null
            storePassword  keystoreProperties['storePassword']
        }
    }
    buildTypes {
        release { signingConfig signingConfigs.release }
    }
}
```

Step 4: Play Console service account

In Google Cloud Console create a service account, grant it access in Play Console under *Users and permissions*, download the JSON key, and store it as a CI secret. Never commit it.

NEVER COMMIT

`key.properties`, `*.jks`, `*.keystore`, the Play service account JSON, and the App Store Connect `.p8` file all belong in `.gitignore` and nowhere else. Once a credential reaches git history, rotate it — deleting the file does not remove it from history.

iOS — the hard one

You need two things to get a build into TestFlight: a **signing identity** (certificate + provisioning profile) and an **authentication method** for the upload. Solve them separately.

Authentication: App Store Connect API key

Do this regardless of which signing approach you pick. Generate a key under *App Store Connect* → *Users and Access* → *Integrations* → *App Store Connect API*. You get three values: a Key ID, an Issuer ID, and a `.p8` file you can download exactly once.

DO NOT USE APPLE ID + PASSWORD

Authenticating with an Apple ID and an app-specific password is legacy. It breaks on two-factor prompts, session expiry, and Apple's periodic re-authentication — all of which surface as a mysterious red build at 2 a.m. The API key is a proper machine credential and doesn't expire on you.

Signing option A — fastlane match (the standard)

Certificates and provisioning profiles are generated once, encrypted, and stored in a private git repository. Every machine — your laptop, a colleague's laptop, the CI runner — pulls the same identities. This is what ends the "works on my Mac" era.

ios/fastlane/Matchfile

```
git_url("git@github.com:yourco/certificates.git") # private repo
storage_mode("git")
type("appstore")
app_identifier(["com.yourco.app"])
```

ios/fastlane/Fastfile

```

default_platform(:ios)

platform :ios do
  desc "Build and upload to TestFlight"
  lane :beta do
    api_key = app_store_connect_api_key(
      key_id:      ENV['ASC_KEY_ID'],
      issuer_id:   ENV['ASC_ISSUER_ID'],
      key_content: ENV['ASC_KEY_P8'],
      is_key_content_base64: true,
    )

    # readonly: CI must never create new certificates
    match(type: 'appstore', readonly: true, api_key: api_key)

    increment_build_number(
      build_number: ENV['BUILD_NUMBER'],
      xcodeproj: 'Runner.xcodeproj',
    )

    build_app(
      workspace: 'Runner.xcworkspace',
      scheme: 'Runner',
      export_method: 'app-store',
    )

    upload_to_testflight(
      api_key: api_key,
      skip_waiting_for_build_processing: true,
    )
  end
end

```

Run `fastlane match appstore` once from your own Mac to populate the certificates repo. From then on CI runs it with `readonly: true`, which guarantees CI can consume identities but never mint new ones — important, because Apple caps how many distribution certificates you can hold.

Signing option B — Codemagic managed signing

Upload your certificate and API key through Codemagic's UI once. It injects signing into the build automatically. There is no `Matchfile`, no certificates repo, no Ruby. See section 6.

Which signing approach to choose

Approach	Best for	Trade-off
fastlane match	Teams of 2+, or anyone who wants signing reproducible on any machine	Ruby toolchain, an extra private repo, an afternoon of setup
Codemagic managed	Solo devs and small teams who want it working today	Signing lives in a vendor UI, not in your repo
Manual certs in secrets	Nobody, really	Works until a cert expires and nobody remembers how it was set up

6. Codemagic — The Low-Pain Alternative

Everything in sections 4 and 5 compressed into one file, because signing and publishing are built in.

```
codemagic.yaml
```



```

workflows:
  ios-release:
    name: iOS Release
    instance_type: mac_mini_m2
    max_build_duration: 60
    environment:
      flutter: stable
      xcode: latest
      cocoapods: default
      groups:
        - appstore_credentials      # env vars defined in the Codemagic UI
    ios_signing:
      distribution_type: app_store
      bundle_identifier: com.yourco.app
    scripts:
      - name: Get dependencies
        script: flutter pub get
      - name: Analyze
        script: flutter analyze --fatal-infos
      - name: Test
        script: flutter test --coverage
      - name: Set build number
        script: |
          echo "BUILD_NUMBER=${($PROJECT_BUILD_NUMBER)}" >> $CM_ENV
      - name: Build IPA
        script: |
          flutter build ipa --release \
            --build-number=$BUILD_NUMBER \
            --export-options-plist=/Users/builder/export_options.plist
    artifacts:
      - build/ios/ipa/*.ipa
      - flutter_drive.log
    publishing:
      app_store_connect:
        auth: integration
        submit_to_testflight: true
        beta_groups:
          - Internal Testers
      email:
        recipients:
          - team@yourco.com
      notify:
        success: true
        failure: true

  android-release:
    name: Android Release
    instance_type: linux_x2
    environment:
      flutter: stable
      java: 17
      android_signing:
        - upload_keystore          # uploaded once via the Codemagic UI
      groups:
        - google_play
    scripts:
      - script: flutter pub get
      - script: flutter analyze --fatal-infos
      - script: flutter test

```

```
- script: |
  flutter build appbundle --release \
    --build-number=$PROJECT_BUILD_NUMBER
artifacts:
- build/**/outputs/bundle/**/*.*aab
publishing:
  google_play:
    credentials: $GCLLOUD_SERVICE_ACCOUNT_CREDENTIALS
    track: internal
    submit_as_draft: false
```

WHY PEOPLE RECOMMEND IT

Compare the iOS workflow above to section 4 plus section 5 combined. No `Matchfile`, no certificates repo, no SSH agent, no Ruby, no Bundler. The `ios_signing` block plus a one-time certificate upload replaces all of it. If your bottleneck is signing rather than pipeline flexibility, that difference is the entire argument.

What you give up

- Your signing setup lives in a vendor's UI rather than in your repository — harder to audit and to reproduce elsewhere.
- Cost past 500 minutes per month.
- Less flexible for non-mobile work; you'll likely still run backend CI somewhere else.

A hybrid worth considering

Run Tier 1 (format, analyze, test) on GitHub Actions where it's free and fast, and let Codemagic handle only the signed iOS and Android release builds. You get cheap fast feedback on every PR and painless signing on release, and you pay Codemagic only for the builds that genuinely need a Mac.

7. Configuration, Flavors & Secrets

Environments with dart-define-from-file

Since Flutter 3.7 this is the correct mechanism. It replaces committed `.env` files and the `flutter_dotenv` pattern for build-time configuration.

env/prod.json

```
{
  "API_BASE_URL": "https://api.yourco.com",
  "SENTRY_DSN": "https://abc123@sentry.io/456",
  "ENVIRONMENT": "production",
  "ENABLE_LOGGING": false
}
```

lib/config.dart

```
class Config {
  static const apiBaseUrl =
    String.fromEnvironment('API_BASE_URL', defaultValue: 'http://localhost:8000');
  static const environment =
    String.fromEnvironment('ENVIRONMENT', defaultValue: 'dev');
  static const enableLogging =
    bool.fromEnvironment('ENABLE_LOGGING', defaultValue: true);
}
```

```
flutter build apk --flavor prod --dart-define-from-file=env/prod.json
```

In CI, write that JSON from secrets at build time rather than committing it:

```
- name: Write environment config
  run: echo '${{ secrets.ENV_PROD_JSON }}' > env/prod.json
```

CRITICAL SECURITY POINT

Anything compiled into the app binary is **extractable**. A determined person can pull every string out of your APK or IPA in minutes. `--dart-define` is for *configuration* — base URLs, feature flags, environment names, public keys.

Real secrets — API secret keys, database credentials, payment provider secrets, signing keys — must never be in the app at all. They belong on your backend, behind an endpoint the app authenticates against. If your Stripe secret key is in your Flutter app, that is a live incident, not a configuration choice.

Flavor setup

Define matching flavors on both platforms so `--flavor dev` works end to end.

android/app/build.gradle

```
android {
    flavorDimensions "env"
    productFlavors {
        dev { dimension "env"; applicationIdSuffix ".dev"; resValue "string", "app_name",
"MyApp Dev" }
        staging { dimension "env"; applicationIdSuffix ".staging"; resValue "string", "app_name",
"MyApp Stg" }
        prod { dimension "env"; resValue "string", "app_name",
"MyApp" }
    }
}
```

On iOS, create matching Xcode *schemes* and *configurations* (Debug-dev, Release-dev, and so on). This is fiddly by hand — the `flutter_flavorizr` package generates it correctly and is worth using.

WHY SUFFIX THE APPLICATION ID

`applicationIdSuffix` lets dev, staging, and production builds coexist on the same device. Testers can compare a release build against a staging build side by side without uninstalling. Pair it with per-flavor app icons and names so nobody files a bug against the wrong build.

Secrets inventory

Secret	Used for	Where it lives
<code>KEYSTORE_BASE64</code>	Android upload key	CI secret
<code>STORE_PASSWORD</code> , <code>KEY_PASSWORD</code> , <code>KEY_ALIAS</code>	Keystore access	CI secret
<code>PLAY_SERVICE_ACCOUNT_JSON</code>	Play Console uploads	CI secret
<code>ASC_KEY_ID</code> , <code>ASC_ISSUER_ID</code> , <code>ASC_KEY_P8</code>	App Store Connect API	CI secret
<code>MATCH_PASSWORD</code> , <code>MATCH_SSH_KEY</code>	fastlane match decryption + repo access	CI secret
<code>ENV_PROD_JSON</code>	Build-time app configuration	CI secret
<code>FIREBASE_TOKEN</code> / service account	App Distribution	CI secret

8. Versioning Strategy

Manual version bumps are a reliable source of rejected uploads. Let CI own the build number.

How Flutter versions work

pubspec.yaml

```
version: 1.2.3+45
#      ^^^^^ ^
#      |     | build number → Android versionCode, iOS CFBundleVersion
#      |_____ version name → Android versionName, iOS CFBundleShortVersionString
```

The **build number** must **strictly increase** with every upload to either store. The version name is what users see and can repeat across builds.

The recommended pattern

Component	Source	Why
Build number	<code>{{ github.run_number }}</code> or Codemagic's <code>\$PROJECT_BUILD_NUMBER</code>	Monotonic by construction. Cannot be forgotten or duplicated.
Version name	The git tag, stripped of its <code>v</code>	One source of truth. The tag <i>is</i> the release.

```
- name: Derive version from tag
  run: |
    VERSION=${GITHUB_REF#refs/tags/v}      # v1.2.3 → 1.2.3
    echo "VERSION=$VERSION" >> $GITHUB_ENV

- run: flutter build appbundle --release
      --build-name=$VERSION
      --build-number=${{ github.run_number }}
```

THE WORKFLOW THIS GIVES YOU

Releasing becomes `git tag v1.2.3 && git push --tags`. Nothing else. No editing `pubspec.yaml`, no forgetting to bump, no "version code 45 has already been used" rejection after a fifteen-minute build.

ONE MIGRATION HAZARD

If you already have builds in the stores, your CI run number will start well below your existing build number and every upload will be rejected. Fix it with an offset — `${{ github.run_number }} + 1000` — chosen to clear your current highest build number. Set it once and leave it alone.

9. Distribution & Release Channels

The channels, and when each applies

Channel	Platform	Use for	Notes
Firebase App Distribution	Both	Internal team builds from <code>main</code>	Fastest path. Free. No review. Testers get an email and install directly. The default for daily builds.
TestFlight	iOS	Beta testing	Internal groups: no review, minutes to arrive. External groups: needs a review, allows up to 10,000 testers.
Play internal track	Android	Beta testing	Near-instant, up to 100 testers. Also: closed, open, and production tracks.
Store production	Both	Public release	Apple review typically 24–48 h. Google review typically hours to a couple of days.
Shorebird	Both	Urgent Dart-only fixes	Over-the-air code push. No store review. Dart code only.

Firebase App Distribution in CI

```
- name: Distribute to internal testers
  uses: wzieba/Firebase-Distribution-Github-Action@v1
  with:
    appId: ${{ secrets.FIREBASE_ANDROID_APP_ID }}
    serviceCredentialsFileContent: ${{ secrets.FIREBASE_SERVICE_ACCOUNT }}
    groups: internal-testers
    file: build/app/outputs/flutter-apk/app-release.apk
    releaseNotes: |
      Branch: ${{ github.ref_name }}
      Commit: ${{ github.sha }}
      ${{ github.event.head_commit.message }}
```

Shorebird — worth understanding

Shorebird lets you push a patched Dart bundle to installed apps without going through store review. When a production crash appears on a Friday evening, the difference between a two-minute patch and a forty-eight-hour Apple review is significant.

```
shorebird release android --flavor prod # cut a patchable release
shorebird patch android --flavor prod # ship a Dart-only fix, no review
```

Its limits

- **Dart code only.** Native changes, new plugins, asset changes, or dependency updates that touch native code all require a full store release.
- Users must have launched the app to receive the patch.
- Apple's guidelines permit it — it doesn't alter the app's core purpose — but treat it as an emergency valve, not your normal release path.

A SENSIBLE RELEASE RHYTHM

Every merge to `main` → Firebase App Distribution, so the team is always on the latest build. Weekly → TestFlight and the Play internal track for wider beta. Every two to four weeks → tagged production release. Shorebird reserved strictly for hotfixes between releases.

10. Testing in CI

Level	Tooling	Speed	When to run
Unit	<code>flutter test</code>	Seconds	Every PR. Non-negotiable.
Widget	<code>flutter test</code> + <code>WidgetTester</code>	Seconds	Every PR.
Golden	<code>matchesGoldenFile</code> , <code>alchemist</code>	Fast	Every PR — but generate goldens in CI, not locally (font rendering differs by platform).
Integration	<code>integration_test</code>	Minutes	<code>main</code> only, or nightly.
E2E	Maestro or Patrol	Slow	Nightly, or pre-release only.

Coverage with a meaningful threshold

```
- run: flutter test --coverage
- name: Strip generated files from coverage
  run: |
    dart pub global activate remove_from_coverage
    dart pub global run remove_from_coverage:remove_from_coverage \
      -f coverage/lcov.info -r '\.g\.dart$' -r '\.freezed\.dart$'
- name: Enforce coverage threshold
  uses: VeryGoodOpenSource/very_good_coverage@v3
  with:
    path: coverage/lcov.info
    min_coverage: 70
```

Stripping generated files matters. Left in, `.g.dart` and `.freezed.dart` files inflate coverage into a number that means nothing.

E2E — Maestro versus Patrol

Tool	Strengths	Choose it when
Maestro	Simple YAML flows, tolerant of timing, very low maintenance, works across frameworks	You want E2E coverage without a maintenance burden. The pragmatic default.
Patrol	Written in Dart, can drive native dialogs — permissions, biometrics, system settings	Your flows depend on native permission prompts or system UI.

```
.maestro/login.yaml
```



```
appId: com.yourco.app
---
- launchApp
- tapOn: "Email"
- inputText: "test@example.com"
- tapOn: "Password"
- inputText: "hunter2"
- tapOn: "Sign In"
- assertVisible: "Welcome back"
```

GATE YOUR SLOW TESTS

Integration and E2E tests are slow and inherently flakier than unit tests. Running them on every PR push produces two outcomes: developers wait, and developers start re-running red builds reflexively without reading them. Put them on a nightly cron and on the pre-release job. Keep the PR gate fast enough that people trust it.

11. Code Quality & Repo Hygiene

Linting

analysis_options.yaml

```
include: package:very_good_analysis/analysis_options.yaml
# or the gentler: package:flutter_lints/flutter.yaml

analyzer:
  exclude:
    - "**/*.g.dart"
    - "**/*.freezed.dart"
    - "lib/generated/**"
  errors:
    invalid_annotation_target: ignore

linter:
  rules:
    public_member_api_docs: false
    prefer_single_quotes: true
```

DO THIS ON DAY ONE

Choose your lint package at the start of a project. Retrofitting `very_good_analysis` onto a fifty-thousand-line codebase produces thousands of violations and a change nobody can review. If you're already deep into a project, adopt it and add per-rule `ignore` entries you burn down over time — don't attempt a big-bang fix.

Dependencies

renovate.json

```
{
  "extends": ["config:base"],
  "schedule": ["before 6am on Monday"],
  "packageRules": [
    {
      "matchManagers": ["pub"],
      "matchUpdateTypes": ["minor", "patch"],
      "groupName": "dart dependencies",
      "automerge": true
    }
  ]
}
```

Weekly grouped updates with auto-merge on green CI. Major updates raise a PR for a human. This keeps you off the "we haven't updated dependencies in eighteen months" cliff, where every upgrade is a breaking one.

Monorepos with Melos

melos.yaml

```
name: my_workspace
packages:
  - packages/**
  - apps/**

scripts:
  analyze:
    run: melos exec -- flutter analyze --fatal-infos
  test:
    run: melos exec --dir-exists=test -- flutter test
  format:
    run: dart format . --set-exit-if-changed
```

Your CI job then reduces to `melos bootstrap && melos run analyze && melos run test` regardless of how many packages exist.

Pre-commit hooks

Catching formatting locally saves a CI round-trip and keeps the diff clean.

.git/hooks/pre-commit

```
#!/bin/sh
dart format --set-exit-if-changed $(git diff --cached --name-only --diff-filter=ACM | grep
'\.dart$') || exit 1
flutter analyze --fatal-infos || exit 1
```

Use the `lefthook` or `husky` equivalents if you want hooks shared across the team rather than per-clone.

12. Monitoring & Crash Reporting

A release pipeline that ships builds but gives you no visibility into what happens next is only half a pipeline.

The symbol upload rule

NON-NEGOTIABLE

Flutter release builds are compiled and obfuscated. Without uploading debug symbols, every production crash report you receive is an unreadable list of memory addresses. **Symbol upload must be part of the release job** — not a manual step someone remembers to do, because they won't.

Building with symbols

```
flutter build appbundle --release \  
  --obfuscate \  
  --split-debug-info=build/symbols
```

Then upload `build/symbols` to your crash reporter in the same job. If you lose those symbol files, crashes from that build are permanently unreadable — treat them as build artifacts worth retaining.

Sentry

```
- name: Upload symbols to Sentry  
  run: |  
    dart pub global activate sentry_dart_plugin  
    dart pub global run sentry_dart_plugin  
  env:  
    SENTRY_AUTH_TOKEN: ${ secrets.SENTRY_AUTH_TOKEN }  
    SENTRY_ORG: yourco  
    SENTRY_PROJECT: flutter-app  
    SENTRY_RELEASE: ${ env.VERSION }+${ github.run_number }
```

Firebase Crashlytics

```
- name: Upload Crashlytics symbols  
  run: |  
    firebase crashlytics:symbols:upload \  
      --app=${ secrets.FIREBASE_ANDROID_APP_ID } \  
      build/symbols
```

Choosing

Tool	Strengths	Consider
Sentry	Excellent Flutter SDK, performance tracing, release health, session tracking, breadcrumbs	Paid past a modest free tier
Crashlytics	Free, unlimited, integrates with the rest of Firebase	Weaker performance tooling; ties you further into Firebase

Tag your releases

Always send the release identifier with crash reports. It's what turns "we have crashes" into "1.2.4 introduced this crash, 1.2.3 was clean" — which is the difference between guessing and knowing.

```
await SentryFlutter.init((options) {  
  options.dsn = Config.sentryDsn;  
  options.release = '${packageInfo.version}+${packageInfo.buildNumber}';  
  options.environment = Config.environment;  
  options.tracesSampleRate = 0.2;  
});
```

13. Performance & Cost Optimisation

Caching — the biggest single win

```
- uses: subosito/flutter-action@v2
  with:
    flutter-version: '3.35.x'
    cache: true                # caches the Flutter SDK itself

- uses: actions/cache@v4      # pub packages
  with:
    path: ~/.pub-cache
    key: pub-${{ hashFiles('**/pubspec.lock') }}

- uses: actions/cache@v4      # CocoaPods (iOS)
  with:
    path: ios/Pods
    key: pods-${{ hashFiles('ios/Podfile.lock') }}

- uses: actions/setup-java@v4 # Gradle (Android)
  with:
    distribution: zulu
    java-version: '17'
    cache: gradle
```

On a typical project this takes a cold ten-minute build down to four or five minutes warm — and on macOS runners, at a 10× multiplier, that saving is real money.

Where the minutes go

Optimisation	Typical saving	Effort
Guard the iOS job with <code>if:</code> so it only runs on <code>main</code> and tags	Enormous — often 70–80% of total spend	One line
Enable all four caches above	40–50% of build time	Ten minutes
Run analyze and test on Ubuntu, never macOS	10× on those steps	Job split
Cancel superseded runs on the same branch	10–20%	One config block
Skip CI on docs-only changes	Varies	<code>paths-ignore</code>

```
concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true

on:
  pull_request:
    paths-ignore: ['**/*.md', 'docs/**', '.gitignore']
```

Self-hosted macOS runners

Once you're consistently over roughly 1,500 macOS minutes a month, a dedicated Mac mini becomes cheaper than cloud minutes within a year — and it's faster, because your caches are warm on local disk rather than restored over the network on every run.

THE REAL COST OF SELF-HOSTING

You inherit the maintenance: Xcode updates, certificate expiry, disk filling with derived data, macOS updates that break the toolchain, and the physical machine's uptime. It's genuinely cheaper only if someone owns it. For a team of one or two, cloud minutes are usually the better trade.

14. Common Pitfalls

1. Using `flutter-version: stable`

A Flutter release you didn't ask for breaks your build at the worst possible moment. **Pin the version.** Bump it deliberately, in its own PR, where you can see what broke.

2. Building iOS on every PR

The 10× macOS multiplier will consume a month's allowance in about two days. Guard it with `if: github.ref == 'refs/heads/main'`.

3. Holding your own Android app signing key

Lose it and you can never update the app again — you'd publish under a new package name and abandon your install base. **Enable Play App Signing.** The upload key is recoverable; the app signing key is not.

4. Committing secrets

Once a credential reaches git history, deleting the file does not remove it. Rotate the credential. Add `key.properties`, `*.jks`, `*.p8`, and service account JSON to `.gitignore` before your first commit, and consider `gitLeaks` in CI.

5. Forgetting to upload debug symbols

Obfuscated release crashes are unreadable without them. Put the upload in the release job. If you lose the symbol files for a shipped build, those crashes are lost permanently.

6. Manual version bumping

Someone forgets, the store rejects the upload, and you discover it fifteen minutes into a build. Derive the build number from the CI run number.

7. Slow PR checks

A PR gate over about five minutes stops being a gate and becomes something people work around. Keep Tier 1 fast and push everything slow to `main` or nightly.

8. Running `fastlane match` without `readonly: true` in CI

CI will happily generate new certificates, and Apple caps how many distribution certificates you can hold. Once you hit the limit, revoking the wrong one breaks every other developer's local signing. Always `readonly: true` on CI.

9. Putting real secrets in `--dart-define`

Anything in the binary is extractable in minutes. Configuration is fine; secret keys are not. They belong behind your backend.

10. Not caching CocoaPods

`pod install` on a cold macOS runner routinely takes three to five minutes. At the 10× multiplier that is thirty to fifty billed minutes per build, for a cache you could have configured in one minute.

11. Ignoring flaky integration tests

A test suite that fails one run in five trains the team to hit re-run without reading the output. At that point it detects nothing. Quarantine flaky tests, fix them, and only then return them to the gate.

12. No build notifications

A red `main` nobody notices until the next morning wastes a day. Wire Slack or email notifications on failure — and only on failure, or people will filter them.

15. Implementation Roadmap

Four weeks, in the order that keeps you unblocked. Attempting all of it in one sitting is the most common way these projects stall.

Week 1 — Foundation

- ❑ Add `analysis_options.yaml` with `flutter_lints` or `very_good_analysis`
- ❑ Fix or explicitly ignore existing violations so the analyzer is green
- ❑ Create `.github/workflows/ci.yaml` with the test job only
- ❑ Pin the Flutter version and enable SDK + pub caching
- ❑ Confirm the PR check runs in under four minutes
- ❑ Make the check required for merge in branch protection settings

Week 2 — Android delivery

- ❑ Enable Play App Signing in Play Console
- ❑ Generate an upload keystore, base64-encode it, store it as a CI secret
- ❑ Wire `key.properties` and the Gradle signing config
- ❑ Add the Android build job, gated to `main`
- ❑ Set the build number from `github.run_number` (with an offset if you have existing builds)
- ❑ Create a Play Console service account and upload to the internal track
- ❑ Add Firebase App Distribution for daily team builds

Week 3 — iOS delivery

- ❑ Generate an App Store Connect API key and store the three values as secrets
- ❑ Decide: `fastlane match` or Codemagic managed signing
- ❑ If match — create the private certificates repo and run `fastlane match appstore` once locally
- ❑ Write the `Fastfile` `beta` lane
- ❑ Add the iOS job on `macos-15`, **gated with** `if:`
- ❑ Cache CocoaPods
- ❑ Verify a build reaches TestFlight end to end

Week 4 — Release & observability

- ❑ Add the tag-triggered release workflow (`v*.*.*`)

- ❑ Derive the version name from the git tag
- ❑ Build with `--obfuscate --split-debug-info`
- ❑ Upload symbols to Sentry or Crashlytics in the release job
- ❑ Tag crash reports with release and environment
- ❑ Add Renovate or Dependabot for `pubspec.yaml`
- ❑ Add build failure notifications to Slack or email
- ❑ Document the release process in `CONTRIBUTING.md` — one page, so it isn't only in your head

Later, when you need it

- Golden tests, generated in CI so font rendering is consistent
- Maestro E2E flows on a nightly cron
- Shorebird for emergency Dart-only hotfixes
- Melos, if the project splits into multiple packages
- A self-hosted Mac runner, once you're reliably past ~1,500 macOS minutes a month

The short version

GitHub Actions for the pipeline. **Fastlane** with `match` for the mobile chores. **App Store Connect API key** for authentication — never an Apple ID. **Play App Signing** so a lost key isn't fatal. **Firebase App Distribution** for daily team builds, **TestFlight** and **Play internal** for beta, **git tags** for production.

Move iOS to **Codemagic** the moment signing starts costing you days rather than hours — that's precisely the problem it exists to solve, and switching is cheap.

Build it in this order: Tier 1 PR checks → Android delivery → iOS delivery → tagged releases. Get each one green and live with it before starting the next.